

二分探索のあれこれ

目次

0.二分探索とは

1.配列の二分探索

2.答えて二分探索

付録:bisectについて

0.二分探索とは

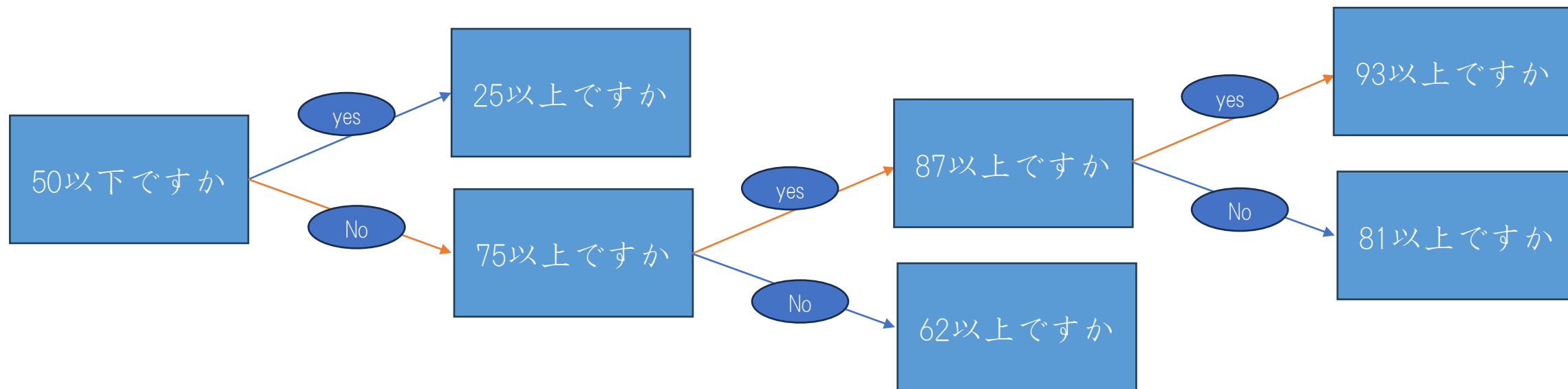
まずは，こういうあたりゲームを考えよう。

私はは1から100までの中の一つの数字を思い浮かべている。
あなたは，その数字を当てることを目指しています。
当たり方については，〇〇以上/以下ですか，〇〇ですかのいずれかを聴く。
目標としては，やり取りの回数を最小化することです。

0.二分探索とは

素朴な方法では，1ですか，2ですか，…になる。しかし，こういう方法では，例えば99の場合になると99回聞かなければならない。それは効率が望ましくない。

しかし，以下のやり方で，確実に6回しかやる必要がない方法が存在する。



0.二分探索とは

つまり，毎回にはあり得る候補の中央値を選び，その値以上なのか以下なのかを聴く。最初には1から100の100種の答えがあり得るが，2回目では50から100の50種しか残されない。そして3回目には25種，4回目は12か13種のように，毎回には範囲の半分だけに残される。

このように，中央で区切ることで，探索する範囲を半分ずつ減らしていきアルゴリズムを二分探索法という。

0.二分探索とは

二分探索のアルゴリズム

left $\leftarrow$ 0

right $\leftarrow$ 答えがあり得る最大値

while(left < right - 1) #leftとrightの間に何もないまで繰り返す:

 mid = (left + right) / 2

 if midでいい:

 return mid

 else:

 if midがちょっと小さい:

 left = mid + 1

 else:

 right = mid

1.配列の二分探索

二分探索を配列に合わせて使うのは，配列でその要素を探すか，挿入するならどこに挿入すればよいかの問題をよく使われている。

例えば，配列 $A = [3, 4, 5, 7, 9, 10, 12, 16, 19]$ に対して，要素10は存在するかという問題を考えてみよう。

もちろん，最初から最後までをfor文で全探索をすれば解ける問題だが，それはすべてのケースを確認しないといけないので効率が良くない。

ここで，二分探索を考える場合，最初にはAの前半には10が存在するか，またはAの後半に10が存在するかを考えればよい。

ここでのAは小さい順で並べているので，10とAの中央にある値を比べばいい。つまり，まんなかの要素9に対して， $10 > 9$ なので後半に存在する可能性はある。

1.配列の二分探索

ここで、二分探索を考える場合、最初にはAの前半には10が存在するか、またはAの後半に10が存在するかを考えよう。

ここでのAは小さい順で並べているので、10とAの中央にある値を比べよう。つまり、まんなかの要素9に対して、 $10 > 9$ なので後半に存在する可能性はある。*

*の操作によって、今探索すべき範囲をAの全体からAの半分になる。さらには[10, 12, 16, 19]の中に10はあるかを探す操作に戻る。すなわち、そのまんなかに存在する12または16(コードの書き方によるが、数が偶数の場合にはどちらの中央値を選んでもかまわない。次からは12を例として説明する)だけ残される。

$10 < 12$ なので10が存在する範囲は[10]だけ、かつ $10 = 10$ なので10はあると判断できる。また、もし11を探すなら、前には同じところに範囲を[10]に定まり、 $11 > 10$ より11が10と12の間に存在するわけがそこには要素がない(添え字の差が1)より11がないと判断できる。そして、もし11をこの配列に挿入すればその添え字が7になるはずこともわかる。

1.配列の二分探索

このように，二分探索を使って，要素を配列に存在するかどうか，または挿入したいならどこに挿入すればいいのかを容易にわかる。

しかし，ここでは一つの注意点がある。それは，探索する配列はソート済みでないといけないことである。

なぜなら，配列をソートしないと，添え字と大小関係についての関係がなくなるからである。例えば， $[3, 1, 4, 2, 5, 9]$ に対して，2を探す場合に，まず4と比べて，4の左側にある可能性だけさらに考えるので，右に存在するケースが抜けてしまう。

1.配列の二分探索

では，次の練習問題を解いてみましょう。

1. [C - Counting 2](#) 素朴な問題で，どこに挿入すればいいのかが感がよう
2. [C - Invisible Hand](#) あり得る答えが有限個があって，それぞれ確認しよう
3. [007 - CP Classes \(★3\)](#) 与えられた要素ともっとも近いものはどうする
4. [D - Sleep Log](#) ちょっと難しいかもしれないが，「累積和」という概念はまだ扱っていないので，余力があれば調べてみてください

2. 答えで二分探索

配列の探索と違い，問題全体の答えがある範囲に定まっている場合にも，二分探索が利用できる。

最初の当たりゲームでは，答えが1から100という範囲に存在することがわかるので，その範囲で探せばよい。

実際には，もっと複雑なケースが存在する。それは，ある範囲内の全部の値も条件を満たすが，その最小値/最大値を求める問題である。

このような問題に対して，その「臨界値」を二分探索で探す。つまり，例えば答えが x のときに条件を満たされば， $\text{right} = x$ とし，そうでないと $\text{left} = x$ とする方法で，結局条件を満たすあり得る最小値のところをが left のところになる。

2. 答えで二分探索

では，次の練習問題を解いてみましょう。

1. [B12 - Equation](#) 関数の単調性から考えよう
2. [D - Minimum Width](#) 毎行の文字数としてあり得る範囲はどうか？
3. [C - MAD TEAM](#) 戦力値として，あり得る範囲が存在する。それと，集合というデータ構造が必要になる

付録:bisectについて

bisectとは，Pythonの二分探索用のライブラリである。主には以下の4つの関数を使われている。

- bisect_left(A, x)
- bisect_right(A, x)
- insort_left(A, x)
- insort_right(A, x)

bisect_*では，もしxをAに挿入したら，xの添え字がいくつになるかを返しが，挿入自体はしない。その代わりにinsort_*は挿入まで行う。

*_leftは，挿入する値と同じ値が存在しても，同じものの左に挿入するが，*_rightは同じものの右に挿入する。